kedd :: CHEATSHEET

Kernel estimator and bandwidth selection for a density and its derivatives — S4 classes, ggplot2 graphics, Rcpp/OpenMP

kedd 2.0.0
A. C. Guidoum & A. Henider
`library(kedd)`

The estimator

For a sample $X_1,\dots,X_n$ from a density f, the kernel estimator of the r-th derivative is

$$\hat{f}_h^{(r)}(x) = {}^1\!/_{n\,h^{r+1}} \sum_i K^{(r)}((x - X_i)/h)$$

$$\mathrm{AMISE} = R(K^{(r)}) \,/\, (n\,h^{2r+1}) + {}^1\!/_4\, \mu_2(K)^2\, h^4\, R(f^{(r+2)})$$

so $h_{opt} \propto n^{-1/(2r+5)}$: **derivatives need larger bandwidths** than the density.

$R(g)=\int g^2$, $\mu_k(K)=\int x^k K(x)dx$.

Workflow

`data x` → `h.ucv(x, r)` → `dkde(…, h)` → `plot()`

```
library(kedd)
x  <- rnorm(300)
bw <- h.ucv(x, deriv.order = 1)  # select h
bw; plot(bw)                     # show(), CV curve
f1 <- dkde(data = x, h = bw@h, deriv.order = 1)
plot(f1)                         # estimate of f'
```

Slots are read with `@` : `bw@h`, `f1@x`, `f1@y`.

Density & derivatives dkde()

dkde(x = NULL, data, h, deriv.order = 0, kernel = "gaussian", nthreads = 0)

- `x` evaluation points (default: 512 points on $[\min - 3h, \max + 3h]$)
- `h` missing ⇒ UCV bandwidth `h.ucv(data, r, kernel)`
- returns class **dkde**: slots `x y h kernel deriv.order n data call`

```
d <- dkde(data = x, h = 0.5, kernel = "biweight")
plot(d, rug = TRUE, band = TRUE, type = "l")
```

Kernels kernel = "…"

name	K(x)	supp.	r_{max}	UCV/MCV/CCV
`gaussian`	$\varphi(x)$	ℝ	any	any
`epanechnikov`	$¾(1-x^2)$	[−1,1]	2	1 / 0 / –
`uniform`	½	[−1,1]	0	0 / – / –
`triangular`	$1-\lvert x\rvert$	[−1,1]	1	0 / – / –
`biweight`	$15/16\,(1-x^2)^2$	[−1,1]	4	2 / 1 / 0
`triweight`	$35/32\,(1-x^2)^3$	[−1,1]	6	3 / 2 / 1
`tricube`	$70/81\,(1-\lvert x\rvert^3)^3$	[−1,1]	9	3 / 2 / 1
`cosine`	$\pi/4\cos(\pi x/2)$	[−1,1]	any	1 / 0 / –
`silverman`	$½e^{-\lvert x\rvert/\sqrt{2}}\sin(\lvert x\rvert/\sqrt{2}+\pi/4)$	ℝ	any	any
`logistic`	$e^{-x}/(1+e^{-x})^2$	ℝ	20	8 / 8 / 8
`pow(s,t)`	$c_{s,t}(1-\lvert x\rvert^s)^t$	[−1,1]	min(st,16)	r*/r*−1/r*−2

Names can be abbreviated (`"epa"`). r_{max}: last non-zero $K^{(r)}$ (kernel.fun, kernel.conv, dkde). UCV (also BCV, TCV, AMISE) / MCV / CCV: highest r with a finite criterion, else `NA` + warning; SCV: any, MLCV: 0; – = not available; r* = largest integer r < t+½ (2r ≤ st if s, t are integers; r < s+½ unless s is even), at most 6.

Power family Pow(s,t) kernel.pow()

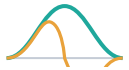
$$K_{s,t}(x) = c_{s,t}(1-|x|^s)^t\, 1_{|x|\le 1},\quad c_{s,t} = s \,/\, (2\,B(1/s, t+1))$$

```
kernel.pow(2, 3)            # "pow(2,3)"
h.ucv(x, kernel = "pow(1.5,2.5)")   # s>0, t≥0
```

`pow(s,0) uniform` `pow(1,1) triangular` `pow(2,1) epanechnikov` `pow(2,2) biweight` `pow(2,3) triweight` `pow(3,3) tricube`

Exact constants: $\mu_k = B((k+1)/s, t+1)/B(1/s, t+1)$; `"pow"` alone = `"pow(2,2)"` .

Kernel functions kfun



kernel.fun(x = NULL, deriv.order = 0, kernel = "gaussian", n.grid = 512) → $K^{(r)}(x)$

```
kf <- kernel.fun(kernel = "biweight", deriv.order = 2)
plot(kf, type = "o", zero.line = TRUE)
```

Default grid: $[-8, 8]$ (gaussian, silverman, logistic), $[-1.1, 1.1]$ otherwise.

Convolutions kconv

kernel.conv(x = NULL, deriv.order = 0, kernel = "gaussian", n.grid = 512) → $(K^{(r)} * K^{(r)})(x)$, the building block of the CV criteria.

```
kc <- kernel.conv(kernel = "logistic", deriv.order = 1)
plot(kc)
kernel.conv(x = 0, kernel = "logistic", 1)@y   # -R(K') = -1/30
```

At 0: $(K^{(r)} * K^{(r)})(0) = (-1)^r\, R(K^{(r)})$. Default grid $[-2.1, 2.1]$ for compact kernels.

Kernel constants

```
Cpp_kernel_constants("biweight", 0)
#      c     mu2     mu4       R
#     NA  0.1429  0.0476  0.7143
Cpp_kernel_constants("pow(1.5,2.5)", 1)  # c, mu2, mu4, R(K')
```

`c` = normalising constant of a pow(s,t) kernel (`NA` for named kernels); `R` = $R(K^{(r)})$.

Datasets ?"kedd-data"

n = 200, Marron & Wand (1992) mixtures: `kurtotic #4` `outlier #5` `bimodal #7` `trimodal #9` `claw #10` `h.mcv(claw)`

kedd :: BANDWIDTH SELECTION

Eight data-driven selectors — all return an S4 object of class **bandwidth**

common arguments
`x, deriv.order = 0, kernel = "gaussian", lower, upper, tol, nthreads = 0`

Selectors

function	method	r	kernels
h.amise	AMISE rule (normal reference)	any	all
h.ucv	unbiased / least-squares CV	any	all
h.bcv	biased CV: `whichbcv = 1` (BCV1) or `2` (BCV2)	any	all
h.ccv	complete CV	any	all
h.mcv	modified CV	any	all
h.tcv	trimmed CV	any	all
h.scv	smoothed CV, pilot `g`	any	gaussian
h.mlcv	maximum-likelihood CV	0	all

```
bw <- h.bcv(x, whichbcv = 2, deriv.order = 1,
            kernel = "triweight")
h.scv(x, g = 0.3)                  # own pilot
h.mlcv(x, kernel = "epanechnikov")
```

Search interval & tolerance

- `optimize()` on $[0.01\ h_{os}, 2\ h_{os}]$, h_{os} = over-smoothing bandwidth (slot `hos`)
- `tol` default `0.01 * lower`
- invalid `lower` / `upper` ⇒ warning, defaults used
- warning "*minimum occurred at one end of the range*" ⇒ inspect `plot(bw)`, widen `lower` / `upper`

```
h.ucv(x, lower = 0.05, upper = 1.5)
```

Graphics ggplot2

Every `plot()` prints a ggplot and returns it invisibly; customise with `main.title`, `x.title`, `y.title`, `colors = list(…)`, `theme = ggplot2::theme_bw()`, `type = "l" | "p" | "o"`.

```
p <- plot(dkde(data = x, h = 0.4))
p + ggplot2::geom_vline(xintercept = 0)
```

Criteria $u = (X_j - X_i)/h$

UCV $R(K^{(r)})/(nh^{2r+1}) + (-1)^r/(n(n-1)h^{2r+1}) \sum_{i\neq j} [K^{(r)}*K^{(r)} - 2K^{(2r)}](u)$

BCV1 / BCV2 AMISE with $R(f^{(r+2)})$ estimated by $K^{(r+2)}*K^{(r+2)}$ (BCV1) or $K^{(2r+4)}$ (BCV2)

CCV UCV + higher-order bias terms using $K^{(2r+2)}$, $K^{(2r+4)}$, μ_2, μ_4

MCV UCV with the $\frac{1}{2}\mu_2 K^{(2r+2)}$ correction

TCV UCV where pairs with $|u| \leq 1/(nh^{2r+1})$ are dropped from the $K^{(2r)}$ term

SCV $R(K^{(r)})/(nh^{2r+1}) + (-1)^r\, 2/(n(n-1)) \sum_{i<j} [\varphi^{(2r)}_{s1} - 2\varphi^{(2r)}_{s2} + \varphi^{(2r)}_{s3}]$, $s_1=\sqrt{(2(h^2+g^2))}$, $s_2=\sqrt{(h^2+2g^2)}$, $s_3=\sqrt{2}$ g; default $g = \hat{\sigma} \cdot 0.9266\ n^{-2/13}$

MLCV maximise $n^{-1}\sum \log \hat{f}_{-i}(X_i)$

AMISE $h = \hat{\sigma}\ [243(2r+1)R(K^{(r)}) \,/\, (35\ \mu_2(K)^2\ n)]^{1/(2r+5)}$

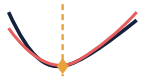
Which selector?

- **UCV**: unbiased but very variable, tends to undersmooth (worse as r grows)
- **BCV, CCV, MCV**: stable, biased towards large h; may have no interior minimum on multimodal data
- **SCV**: good compromise for Gaussian kernels
- **AMISE**: instant rule of thumb, good for unimodal data
- Compare several criteria on one plot (right) before deciding

The bandwidth object

slot	content
`h`	selected bandwidth
`min.obj`	criterion value at `h`
`hos`	over-smoothing bandwidth
`method`	"UCV", "BCV1", …, "MLCV"
`kernel`, `deriv.order`, `data`	inputs used

Plotting criteria plot() / lines()



`plot(bw)` draws the criterion around `h` and marks the minimum; returns `list(h, cv, plot)` invisibly.

```
plot(bw, show.hos = TRUE, log_scale_x = TRUE,
     h = seq(0.1, 1, length.out = 100))
p <- plot(h.ucv(x))
p$plot + lines(h.bcv(x), color = "darkred") +
         lines(h.mcv(x), color = "orange")
```

`lines()` returns a **ggplot2 layer**, added with `+`. Not available for AMISE (closed-form rule).

Speed & low level

- All $O(n^2)$ criteria run in C++ with OpenMP: `nthreads = 0` all cores, `k` = k threads, `1` sequential; per call, capped by `OMP_THREAD_LIMIT`, identical result. `Cpp_omp_threads(0)` = threads used
- `Cpp_kernel_deriv(k, u, r)`, `Cpp_kernel_conv(k, u, r)`, `Cpp_h_ucv(h, k, r, x, nthreads)`, `Cpp_plot_cv(method, h, k, r, x)`, `Cpp_hos(k, r, n)` — see `?"kedd-lowlevel"`

Learn more

```
vignette("kedd", package = "kedd")
demo("kedd"); demo("kernels"); demo("bandwidths")
?kedd; ?kernel.pow; ?h.ucv
```